多维触觉灵巧手 DexH13 SDK 说明手册 V1.1.0

修 订 页		
版本号	修订（内容）说明	修订时间
V1.0	C++、python 版本上线	2025/01/10
V1.1.0	硬件增加磁编，对应增加位置初始化接口	2025/04/18

帕西尼感知科技(深圳)有限公司

PaXini

多维触觉灵巧手 DexH13

SDK 说明手册

帕西尼感知科技

目录

1. 引言
2. **SDK 简介**
3. **SDK 使用须知**

4. **C++版本使用说明**
5. **Python 版本使用说明**
6. **示例程序**
7. **常见问题**
8. **产品使用注意事项**
9. **总结**
10. **版权声明**

帕西尼感知科技

一、 引言

1.1 目的

随着科技的飞速发展，机器人和自动化技术在各个领域得到了广泛应用。在工业制造、医疗康复、服务机器人等诸多场景中，对机器人的灵活性和精细操作能力的需求日益增加。其中，灵巧手作为机器人系统中的一个重要组成部分，扮演着至关重要的角色。

灵巧手产品是基于高精度触觉传感器、先进的控制算法和精密机械设计相结合的产物，旨在模拟和超越人类手部的灵活性和精细操作能力。它不仅能够完成传统工业机械手所能执行的重复性任务，还具备高灵活度和多自由度的特点，可以应对更多复杂、多变的操作环境。

本使用手册将详细介绍维触觉灵巧手 DexH13 的 SDK 的功能、使用流程、接口设计等。我们希望通过 SDK 手册，为用户提供一个高效、可靠且智能化的解决方案，助力各行业实现更高效、更精准、更柔性的操控作业。

感谢您选择帕西尼的灵巧手产品。我们期待与您共同探索更多应用场景，推动具身智能与人形机器人行业的发展。

1.2 适用范围

本《多维触觉灵巧手 DexH13 SDK 使用手册》文档适用于使用帕西尼多维触觉灵巧手 DexH13 的所有客户使用。

二、SDK 简介

本 SDK 提供 C++及 python 版本，支持在 Linux 环境下运行，基于 Modbus 通讯协议，设计遵循简单易用的原则，同时提供丰富的例程，有效减少客户开发时间，提升开发效率。本 SDK 开放了设备连接、参数设置、视觉、触觉传感器等多个模块接口，同时支持状态查询、多模式控制等功能，本使用手册将会详细介绍各模块接口；

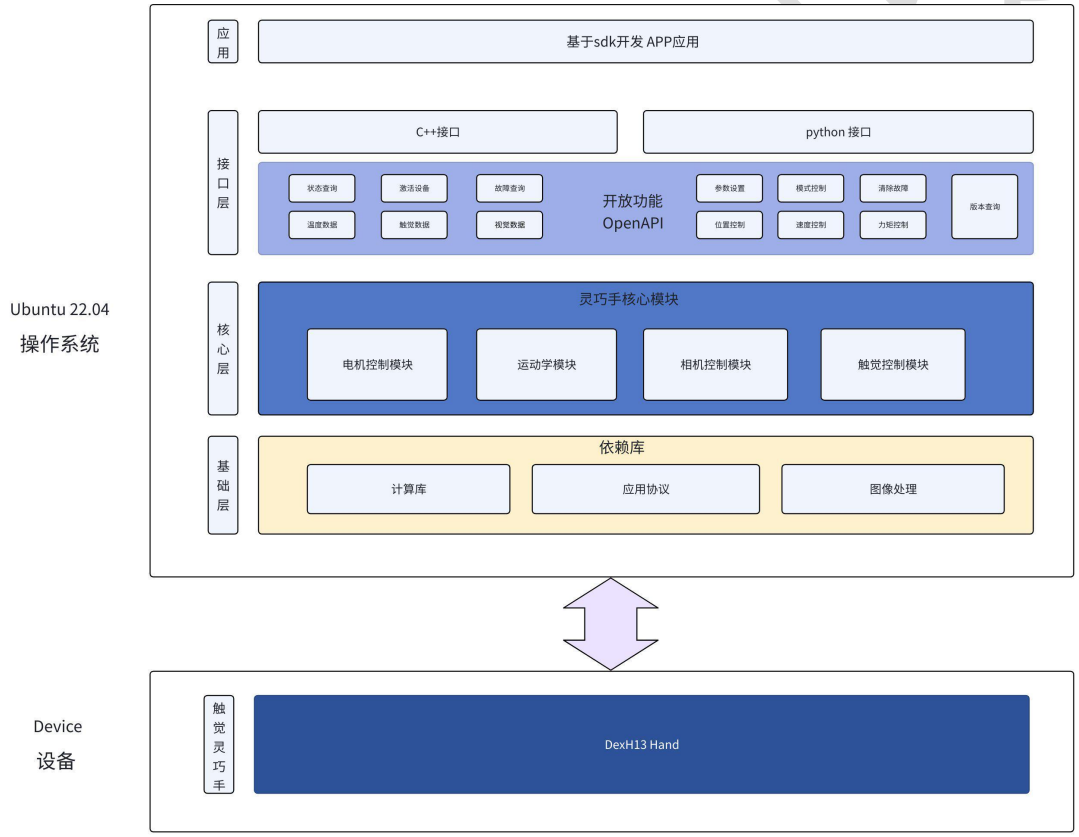


图 1 DexH13 SDK 架构

三、SDK 使用须知

3.1 快速安装

- 推荐环境

```LaTeX
操作系统：Ubuntu 22.04 LTS
CPU：64bit 处理器
内存：4 GB RAM 及以上
存储：20 GB 可用空间及以上
C++      17 标准
python   3.10.12
CMake    >= 3.10
Boost    1.74.0
OpenCV   4.5.4
spdlog   1.9.2
fmt      8.1.1
pybind11 2.13.6
```

- 依赖库安装

```Shell
sudo apt update && sudo apt install -y \
build-essential \
libboost-all-dev \
libopencv-dev \
libspdlog-dev \
libfmt-dev \
libgoogle-glog-dev \
libopenblas-dev \
libdlib-dev \
libjsoncpp-dev \ l
ibfltk1.3-dev \
libyaml-cpp-dev \
python3-pip \
python3-dev \
cmake \
vim \
git

# gsl 安装
git clone https://github.com/ampl/gsl.git
cd gsl
./configure CFLAGS="-g -O0"
make -j
sudo make install

# eigen 安装
```

```
wget https://gitlab.com/libeigen/eigen/-/archive/3.3.7/eigen-3.3.7.tar.gz
tar -xzf eigen-3.3.7.tar.gz
cd eigen-3.3.7
mkdir build
cd build
cmake ..
sudo make install
```

- SDK 安装

```
Bash
sudo dpkg -i DexHandSDK-1.1.0-Linux.deb
python3 -m pip install pxdex-1.1.0-cp310-cp310-linux_x86_64.whl
```

3.2 术语定义

- 普指：食指、中指、无名指的统称。
- 驱动电机关节空间：灵巧手各驱动电机角度值所描述的空间。
- 等效串联关节空间：灵巧手各串联关节角度值所描述的空间。
- 错误码（Error Code）：用于表示 SDK 操作或灵巧手状态中出现的错误类型和原因。

3.3 灵巧手基础信息

3.3.1 产品外观及触觉传感器分布

触觉传感器在灵巧手上的分布如下图所示，共 1140 个测点， 每个触觉测量点可以输出该测量点在 XYZ 三个方向上感受到的力。

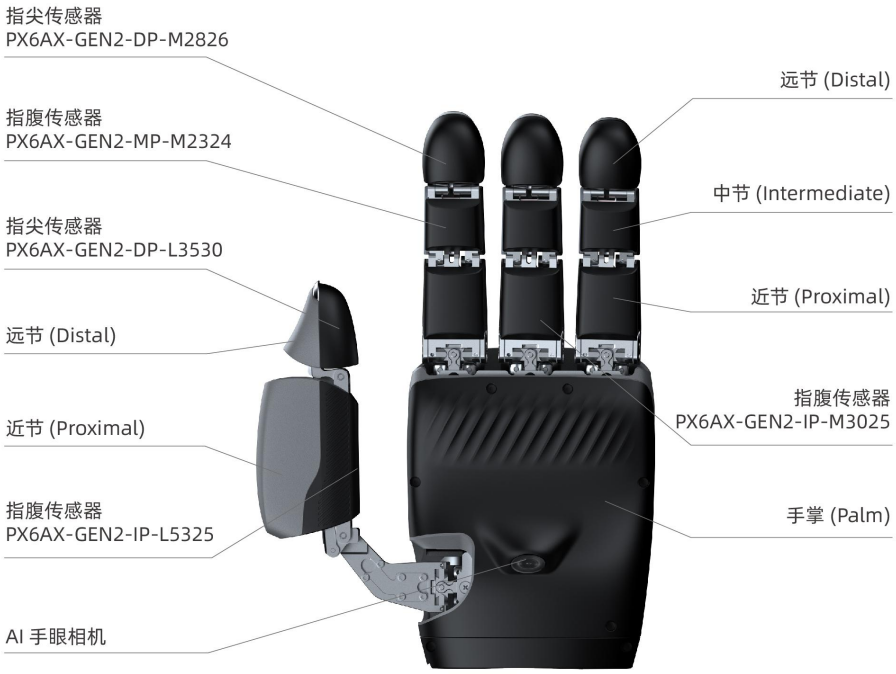


图 2 灵巧手外观及传感器分布

3.3.2 灵巧手操作流程说明

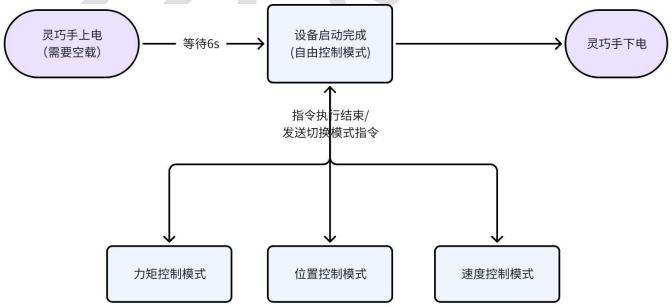


图 3 灵巧手操作流程

注:

- 灵巧手上电需要等待 6s，才能进行后续操作，否则可能无法正确执行指令。
- 建议上电后，执行 initMotorPosition()，初始化电机位置。

3.4 运动控制说明

3.4.1 关节空间坐标系运动控制

- DexH13 灵巧手有 13 个电机，电机编号方式如下:

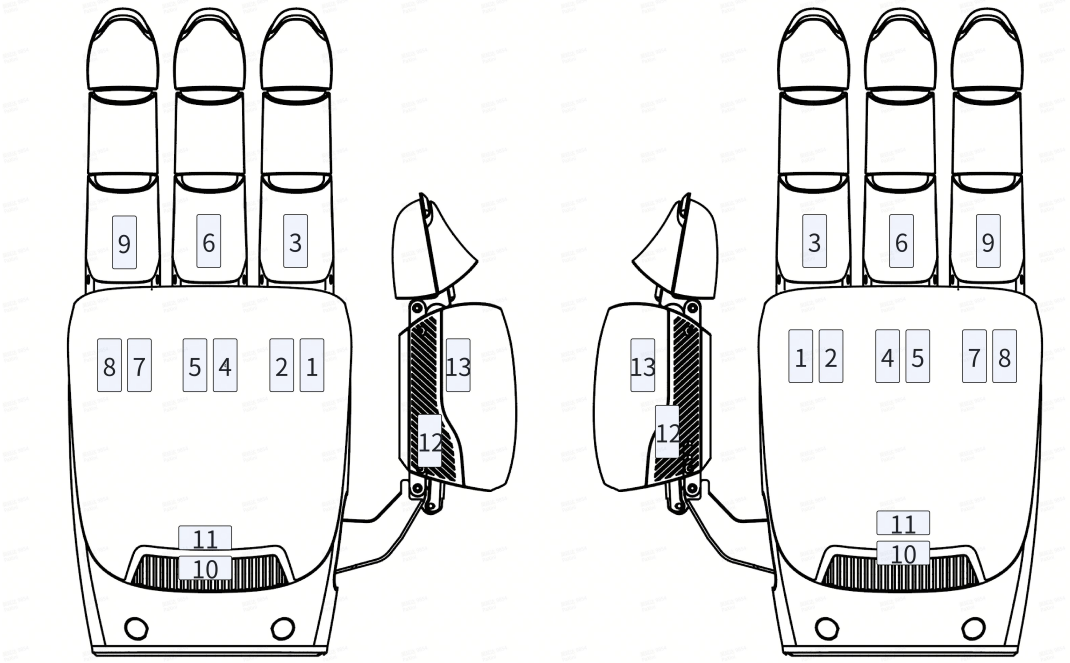


图 4 灵巧手电机顺序编号

3.4.2 触觉传感器信息

- DexH13 灵巧手有 11 个触觉传感器
- 11 个传感器 sensor1~sensor11 编号顺序为：食指底部、食指中间、食指指尖、中指底部、中指中间、中指指尖、无名指底部、无名指中间、无名指指尖、拇指底部、拇指指尖，如图 5 所示：

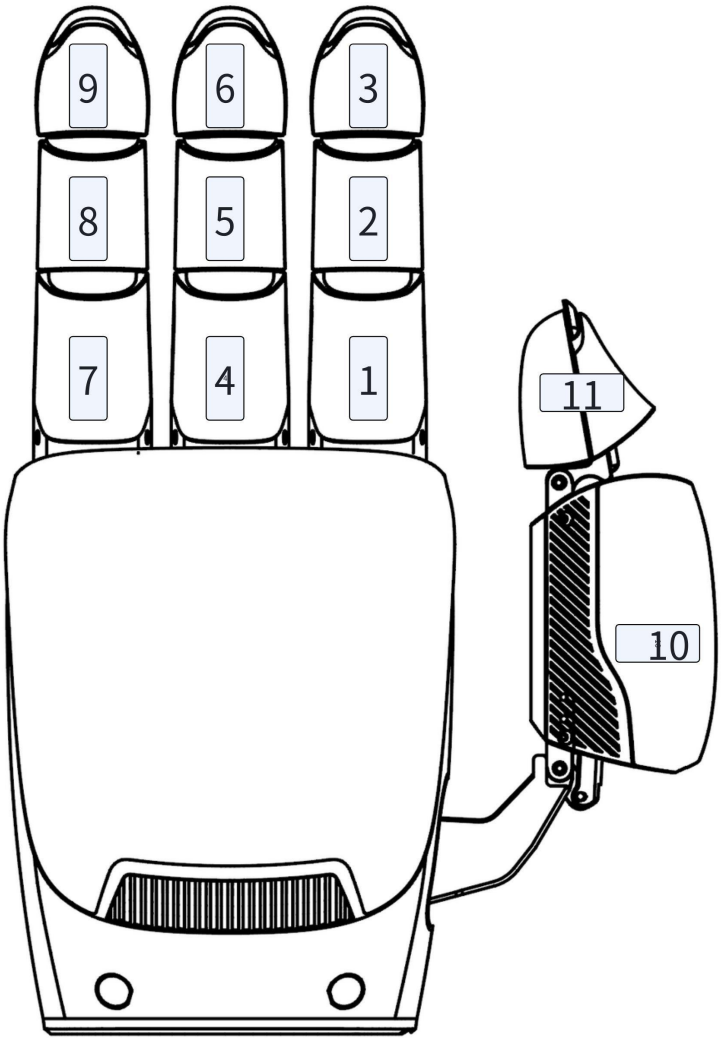


图 5 灵巧手传感器顺序编号

- 触觉传感器坐标说明

在传感器的硅胶尾部底面中点上定义了全局的坐标系（0,0,0），方便确定阵列内各个测点的空间位置。

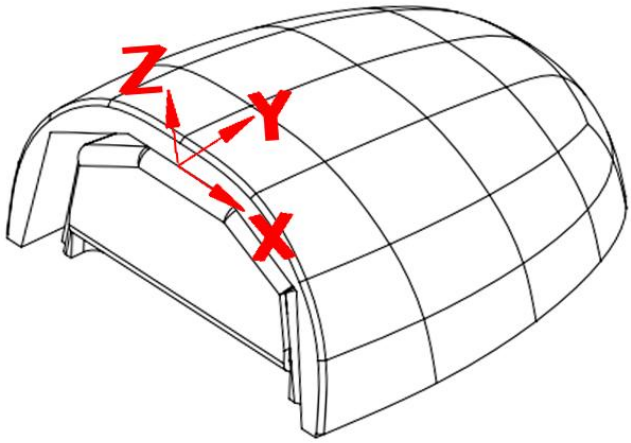
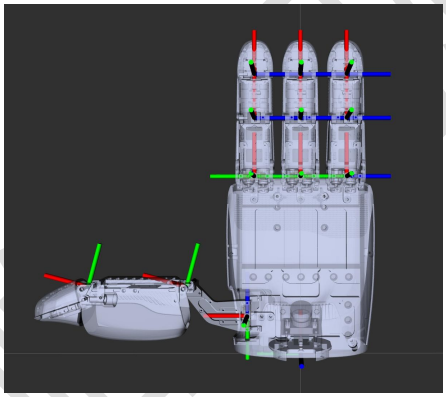
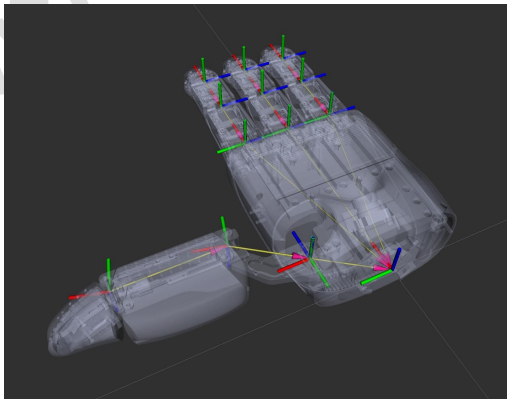


图 6 拇指指尖传感器全局坐标示意

3.4.3 URDF 坐标系

掌内空间坐标系运动控制意在提供用户以灵巧手手腕为参考坐标系控制四指指尖接触点在掌内运动，URDF 中具体坐标如图所示

	手掌坐标系	
左手		
	正视图	侧视图

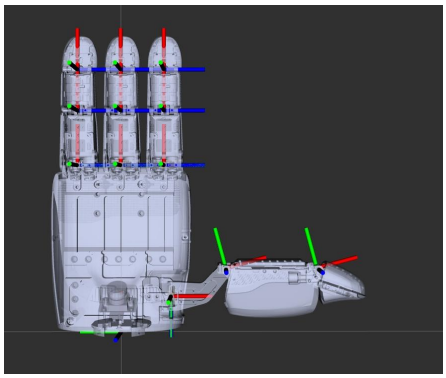
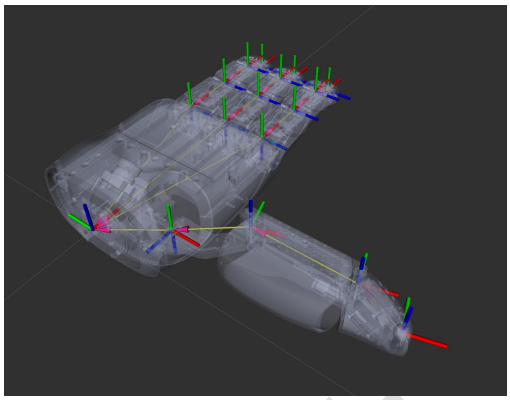
右手	 正视图	 侧视图

图 7 灵巧手坐标系及运控零位示意

注：

1. 整手的坐标原点在法兰中心，坐标系符合右手法则，坐标系颜色 R,G,B 对应 X,Y,Z 坐标。
2. 手指 MCP 侧摆坐标系存在重合，向大拇指方向侧摆为正方向，大拇指 joint 方向向下为正。
3. DexH13 的普指 PIP 关节和 DIP 关节存在耦合关系，URDF 中无法添加此种复杂非线性耦合关系。
4. 上图所示关节位置处于运控零位，三指关节坐标系原点处于同一平面且坐标轴平行，同时平行于拇指 MCP 关节坐标。

3.5 电机控制模式

电机控制模式通常包括多种方式，以适应不同的应用需求。

3.5.1 自由控制模式

自由控制模式（或称无力模式）是电机没有使能的情况下的一种状态， 在此状态下，电机可以在被外力干预下进行自由旋转。这种模式情况下电机没有使能，具体包括系统配置或测试阶段、系统停止对电机控制阶段以及拖动示教阶段等，在此模式下，电机不会受到任何控制信号的限制，可以根据外部条件自由被拖动。

注：下使能即可切换到该模式。

3.5.2 位置控制模式

位置控制模式用于精确控制关节角度或位置，适合于需要高精度定位的应用。在此模式下，关节会根据设定的目标位置进行调整，直至达到目标位置。具体运动方式是一种基于速度为梯形的点到点的运动方式，具体梯形的最大速度和加速度，请参考相关

协议。

3.5.3 速度控制模式

速度控制模式以用户给定的目标速度作为输入，用户可以随时修改目标速度，电机会根据设定的加(减)速度尽可能快的趋近于该目标速度，若在趋近该目标速度时候，目标速度进行了更新，则重新趋近新的目标速度。在此模式下，电机将根据设定的目标速度进行旋转，不再关心具体位置。控制系统会实时调整电机的输入电流，以确保电机以稳定的速度运转。在灵巧手实际应用过程中，请谨慎使用该模式。

3.5.4 力矩控制模式

力矩控制模式是以用户给定的电流作为目标力矩的输入，用于直接控制电机的输入电流，通常用于需要精确控制力矩的应用场景。在此模式下，电机的输出力矩与输入电流成比例，因此，通过设定目标电流，可以精确控制电机的输出力矩。

3.6 错误反馈与处理

3.6.1 错误码

表 2 错误码及释义

错误名	错误码	错误描述	解释说明	解决办法
OVER_VOLT	0	手指电机过压	驱动供电电压高于15V，持续 2 秒	电压恢复后，执行清除错误指令即可
UNDER_VOLT	1	手指电机欠压	驱动供电电压低于10V，持续 2 秒	电压恢复后，执行清除错误指令即可
OVER_CURRENTU	2	手指电机 U 相过流	U 相电流过大	电流恢复正常后，执行清除错误指令即可
OVER_CURRENTY	3	手指电机 V 相过流	V 相电流过大	电流恢复正常后，执行清除错误指令即可
OVER_CURRENTW	4	手指电机 W 相过流	W 相电流过大	电流恢复正常后，执行清除错误指令即可

OVER_TEMPERATURE	5	手指电机过温	电机温度过高	等待手指电机温度恢复到正常范围后，执行清除错误指令即可
ADC_CALIB	6	手指电机硬件自检错误	AD 静态偏差+10%	1、建议检查驱动附件的磁场环境，排除干扰后，断电重启即可 2、进行 1 检查后，还是无法无法消除，建议更换驱动板
FOC_ERR	7	手指电机 FOC 计算错误	无	无
OPD_IQ	8	手指电机 IQ 过流	电机 IQ 电流过大	IQ 电流过流消除后，执行清错执行即可
OPD_U	9	手指电机 U 相缺相	上电初始化时候进行的检测	建议检查电机 U 相线是否有断开，恢复后，执行清错执行即可
OPD_V	10	手指电机 V 相缺相	上电初始化时候进行的检测	建议检查电机 V 相线是否有断开，恢复后，执行清错执行即可
OPD_W	11	手指电机 W 相缺相	上电初始化时候进行的检测	建议检查电机 W 相线是否有断开，恢复后，执行清错执行即可
POS_REF_OVER	12	手指电机位置超上限	无	无
POS_REF_UNDER	13	手指电机位置超下限	无	无
ABZ_ERR	14	手指电机编	光编码一圈丢失计	电机端编码器丢数，需要排查编码器，断电重

帕西尼感知科技

		码器异常	数大于 0.25%	启后，观察现象，一直报错则需返厂
ABZ_Z_ERR	15	手指电机编码器 Z 相异常	初始化时候寻不到 Z 相	电机端编码器 Z 相错误，需要排查编码器，断电重启后，观察现象，一直报错则需返厂
DISCONNECT	31	手指电机未连接	手指驱动和控制板的通讯断开	连接后，系统会自动清错并恢复

3.6.2 错误码清除

电机故障码清除接口，clearFaultCode（），只能清除表 2 中明确执行清除错误指令可清除的错误码。

四、C++版本 SDK 使用说明

4.1 API 接口

DexH13SDK 提供丰富的灵巧手参数设置、控制模式、运行状态、目标位置控制等开发，各模块功能接口说明如下

表 3 接口目录

功能名	说明
DexH13Control	类型定义：创建灵巧手对象（单手模式）
activeHandy	连接设备
disconnectHandy	连接断开
isConnectHandy	连接状态
initMotorPosition	初始化电机位置
setMotorControlMode	设置 -- 自定义电机控制模式

帕西尼感知科技

setMotorMaxCurrent	设置 -- 自定义电机最大电流
setMotorMaxSpeed	设置 -- 自定义电机最大速度
enableMotor	使能 -- 手指电机使能
disableMotor	使能 -- 手指电机下使能
isMotorEnabled	使能 -- 手指电机使能状态
setJointPositionsAngle	控制 -- 手指关节角度控制
setJointPositionsRadian	控制 -- 手指关节弧度控制
setMotorTargetCurrent	控制 -- 电机目标电流
setMotorTargetSpeed	控制 -- 电机目标速度
calibrateSensor	标定 -- 灵巧手触觉传感器（校准操作）
getJointPositionsAngle	查询 -- 获取当前手指关节角度
getJointPositionsRadian	查询 -- 获取当前手指关节弧度
getMotorControlMode	查询 -- 控制模式
getFingerTactile	查询 -- 手指触觉传感器合力信息
getFingerTactileDetail	查询 -- 手指触觉传感器阵列力详细信息
getTemperature	查询 -- 传感器温度数据
getAllTactileData	查询 -- 全部数据（包括合力、阵列力和温度）
isFault	故障 -- 手指故障状态
getFaultCode	故障 -- 当前故障原因
clearFaultCode	故障 -- 清除故障码

帕西尼感知科技

getFrame	图像 -- 当前相机一帧图像
setCameraConfig	图像 -- 相机参数，包括分辨率和帧率
undistortImage	图像 -- 校正，去畸变的图像
getIntrinsicMatrixAndDistCoeffs	图像 -- 获取相机内参矩阵和畸变系数
getHandEyeMatrix	图像 -- 获取手眼相机外标定参数
getSDKVersion	版本 -- SDK 的版本信息
getFirmwareVersion	版本 -- 灵巧手固件版本信息

4.2 详细描述

4.2.1 类定义

请使用命名空间规则 初始化 DexH13Control 对象

命名空间	灵巧手类名
namespace paxini::bot::dexh13	DexH13Control

4.2.2 灵巧手控制

表 3 灵巧手控制详细接口说明

函数名 Function	参数 Parameters	返回值 Return	功能描述 Description
int activeHandy(const std::string &handy_port_num, const std::string &camera_port_num = "none")	handy_port_num: 灵巧手的设备名称 camera_port_num: 相机是设备名称	成功：返回灵巧手的激活状态，1 表示激活左手，2 表示激活右手 失败：终止程序	扫描设备激活灵巧手，Linux 系统下灵巧手设备名可能是 /dev/ttyUSB0，相机设备名可能是 /dev/video0 或其他，请根据实际情况传参，只使用参数 1

帕西尼感知科技

			时，相机默认不开
int disconnectDevice()		成功：返回 0	连接断开
bool isConnectHandy()		成功：返回 1 失败：返回值 0	连接状态
bool setMotorControlMode(const ControlMode mode)	mode: 手指电机控制模式	成功：返回 1 失败：返回值 0	设置 -- 自定义手指电机控制模式，电机控制模式 ControlMode： NONE 无效值、TORQUE_CONTROL_MODE 力矩控制模式、SPEED_CONTROL_MODE 速度控制模式、POSITION_CONTROL_MODE 位置控制模式
bool setMotorMaxCurrent(uint16_t max_current)	max_current: 手指电机的最大电流	成功：返回 1 失败：返回值 0	设置 -- 自定义手指电机最大电流，范围 [0,500], 单位 mA
bool setMotorMaxSpeed(uint16_t max_speed)	max_speed: 电机的最大速度	成功：返回 1 失败：返回值 0	设置 -- 自定义电机最大速度，范围 [0,24000], 单位 rpm
bool enableMotor()		成功：返回 1 失败：返回值 0	使能 -- 手指电机使能，所有的灵巧手指电机
bool disableMotor()		成功：返回 1 失败：返回值	使能 -- 手指电机下使能，所有的灵巧手指

相思鸟感知科技

		0	电机
bool isMotorEnabled()		成功：返回 1 失败：返回值 0	使能 -- 手指电机使能状态，查询手指电机的使能状态
int setJointPositionsAngle(std::vector<FingerAngle>& joint_angles)	joint_angles 数组：包含 4 个手指限位 joint_angles 顺序为： 数组元素 1：食指角度 indexFinger 数组元素 2：中指角度 middleFinger 数组元素 3：环指角度 ringFinger 数组元素 4：大拇指角度 thumb	成功：返回 1 失败：返回值 0	控制 -- 手指关节角度控制，所有 FingerAngle 手指关节（食指、中指、环指、拇指）的关节空间范围： joint1 [-20°，+20°] 关节 1 侧摆角度（掌指双轴关节） joint2 [0°, 90°] 关节 2 屈伸角度（掌指双轴关节） joint3 [0°, 90°] 关节 3 屈伸角度（近端关节） joint4 [0°, 90°] 关节 4 屈伸角度（末端关节）
int setJointPositionsRadian(std::vector<double>& serial_rad)	serial_rad 数组：包含 16 个关节弧度限位 serial_rad 顺序为： index_joint1, index_joint2, index_joint3, index_joint4, middle_joint1, middle_joint2, middle_joint3,	成功：返回 1 失败：返回值 0	控制 -- 手指关节弧度控制，所有手指关节的关节空间范围： joint1 [Pi/180° * -20°, Pi/180° * 20°]（掌指双轴关节-侧位摆动范围） joint2 [0, Pi/180° * 90°] (掌指双轴关节-弯曲伸展范围) joint3 [0, Pi/180° * 90°] (近端关节-弯曲

帕西尼感知科技

	middle_joint4, ring_joint1, ring_joint2, ring_joint3, ring_joint4, thumb_joint1, thumb_joint2, thumb_joint3, thumb_joint4		伸展范围) joint4 [0, Pi/180° * 90°] (末端关节-弯曲伸展范围) 弧度公式：Pi/180° * 角度
int setMotorTargetCurrent(const Dex13MotorCurrent & target_current)	target_current: 电机的目标电流	成功：返回 1 失败：返回值 0	控制 -- 手指关节电流控制，设置手指所有关节电机的目标电流 目标电流范围：[-400, 400] 单位 mA, (峰值不能长时间使用) 一般使用时的范围为 [-200, 200]
int setMotorTargetSpeed(const Dex13MotorSpeed& target_speed)	target_speed: 电机的目标速度	成功：返回 1 失败：返回值 0	控制 -- 手指关节速度控制，设置所有灵巧手指电机的目标速度 目标速度范围：[0, 24000]，单位 rpm （只在速度模式下需要设置）
bool calibrateSensor()		成功：返回 1 失败：返回值 0	标定 -- 灵巧手传感器（校准操作）
ControlMode getMotorControlMode()		成功：返回实际的控制模式 失败：返回 0 无模式	查询 -- 手指电机控制模式，enum ControlMode { NONE = 0, TORQUE_CONTROL_MODE = 1,

帕西尼感知科技

			SPEED_CONTROL_MODE = 2, POSITION_CONTROL_MODE = 3, }
void getJointPositionsAngle(std::vector& joint_angles)	joint_angles 数组：包含 4 个手指限位 joint_angles 顺序为： 数组元素 1：食指关节角度 indexFinger 数组元素 2：中指关节角度 middleFinger 数组元素 3：环指关节角度 ringFinger 数组元素 4：大拇指关节角度 thumb		查询 --获取当前手指关节角度，所有 FingerAngle 手指关节（食指、中指、环指、拇指）的关节空间范围： joint1 [-20°，+20°] 关节 1 侧摆角度（掌指双轴关节） joint2 [0°, 90°] 关节 2 屈伸角度（掌指双轴关节） joint3 [0°, 90°] 关节 3 屈伸角度（近端关节） joint4 [0°, 90°] 关节 4 屈伸角度（末端关节）
void getJointPositionsRadian(std::vector& serial_rad)	serial_rad 数组：包含 16 个关节弧度限位 serial_rad 顺序为： index_joint1, index_joint2, index_joint3, index_joint4, middle_joint1, middle_joint2, middle_joint3, middle_joint4,		查询 -- 获取当前手指关节弧度， 所有手指关节的关节空间范围： joint1 [Pi/180° * -20°, Pi/180° * 20°]（掌指双轴关节-侧位摆动范围） joint2 [0, Pi/180° * 90°] (掌指双轴关节-弯曲伸展范围) joint3 [0, Pi/180° *

帕西尼感知科技

	ring_joint1, ring_joint2, ring_joint3, ring_joint4, thumb_joint1, thumb_joint2, thumb_joint3, thumb_joint4		90°] (近端关节-弯曲伸展范围) joint4 [0, Pi/180° * 90°] (末端关节-弯曲伸展范围) 弧度公式：Pi/180° * 角度
bool isFault()		成功：返回 1 失败：返回值 0	故障 -- 手指故障状态
std::vector getFaultCode()		返回：当前出现的故障的错误码	故障 -- 当前故障原因，struct FingerMotorsError {std::string fingers_name; // 手指名称 ErrorCode error_code; // 手指电机故障码，详见 define 文件定义；}
bool clearFaultCode()		成功：返回 1 失败：返回值 0	故障 -- 清除故障码
const std::string getSDKVersion()		成功：返回 SDK 的版本信息 失败：返回空字符串	版本 -- SDK 的版本信息
std::string getFirmwareVersion()		成功：返回灵巧手的固件版本信息 失败：返回空字符串	版本 -- 灵巧手的固件版本信息

帕西尼感知科技

int initMotorPosition()		成功：返回 1 失败：返回 0	用于初始化电机位置，建议在刚上电时调用，最大耗时约：5s

4.2.3 灵巧手相机

表 4 灵巧手相机详细接口说明

函数名 Function	参数 Paramaters	返回值 Return	功能描述 Description
cv::Mat getFrame();		成功：以 cv::Mat 格式返回当前图像帧。 失败：返回空的 cv::Mat。	获取相机当前拍摄的一帧图像
bool setCameraConfig(int width, int height, float fps);	1. width：图像宽度 2. height：图像高度 3. fps：帧率	成功：返回 true。 失败：返回 false，设置不生效。	设置相机参数，包括分辨率和帧率。只支持 640x480、1280x720、1920x1080 分辨率。帧率只支持 30fps 以及 1920x1080 分辨率下的 60fps。
cv::Mat undistortImage(const cv::Mat &distortedFrame, const cv::Mat &cameraMatrix, const cv::Mat &distCoeffs);	1. distortedFrame：畸变图像 2. cameraMatrix：相机内参矩阵(3x3 的 cv::Mat) 3. distCoeffs：畸变系数(1x4 的 cv::Mat)	成功：返回去畸变的图像。 失败：返回空的 cv::Mat。	图像去畸变。
std::pair<cv::Mat,	1. width：图像宽	成功：以	获取指定分辨率下相

帕西尼感知科技

cv::Mat> getIntrinsicMatrixAndDistCoeffs (int width, int height);	度 2. height：图像高度	std::pair 的格式返回指定分辨率下的相机内参矩阵和畸变系数。 其中内参矩阵是 3x3 的 cv::Mat,畸变系数是 1x4 的 cv::Mat。 失败：终止程序	机内参矩阵和畸变系数。只支持 640x480、1280x720、1920x1080 分辨率。
cv::Mat getHandEyeMatrix();		成功：返回相机手眼标定参数。 失败：终止程序	获取相机手眼标定的转换矩阵（相机到灵巧手基座）。

4.2.4 灵巧手触觉传感器

表 5 灵巧手触觉详细接口说明

函数名 Function	**参数 Paramaters**	**返回值 Return**	**功能描述 Description**
std::vector getFingerTactile()		成功： 返回手指触觉传感器的合力信息 失败： 返回空对象	查询 -- 获取手指触觉传感器信息，手有 11 个传感器，总共 11 个合力，每个传感器只有一个合力信息，合力包含 3 个数据，分别是 x、y、z ForcePoint: 手指触觉传感

帕西尼感知科技

			器信息，内部包含 3 个数据分别是 x: 传感器 x 轴数据 y: 传感器 y 轴数据 z: 传感器 z 轴数据
const std::vector getFingerTactileDetail()		成功：返回手指触觉传感器的阵列力信息 失败：返回空对象	查询 -- 获取手指触觉传感器详细信息，手有 11 个传感器，总共 1140 分布力，分布力包含 3 个数据，分别是 x、y、z 其中食指中部，中指中部，环指中部的触觉传感器有 60 个分布力，其余的关节上触觉传感器有 120 个分布力 ForcePoint: 手指触觉传感器信息，内部包含 3 个数据分别是 x: 传感器 x 轴数据

帕西尼感知科技

			y: 传感器 y 轴数据 z: 传感器 z 轴数据
std::vector getTemperature()		成功：返回手指触觉传感器的温度信息 失败：返回空对象	查询 -- 获取手指传感器温度数据，手有 11 个传感器，总共 11 个温度数据 FingerTemp: 传感器温度数据结构内部包含 std::string finger_name 手指名称 int sensor_ID 传感器 ID float temperature 传感器温度数据
void getAllTactileData(std::vector &finger_tac, std::vector &finger_tac_detail, std::vector &temperature)	finger_tac: 手指触觉传感器的合力信息 finger_tac_detail: 手指触觉传感器的阵列力信息 temperature：手指触觉传感器的温度信息		查询 -- 灵巧手的手指触觉传感器全部数据

五、python 版本 SDK 使用说明

帕西尼感知科技

5.1 API 接口

python 版本是在 c++接口基础上实现，各模块功能接口说明如下

表 6 Python 接口目录

功能名	说明
DexH13Control	类型定义：创建灵巧手对象（单手模式）
activeHandy	连接设备
disconnectHandy	连接断开
isConnectHandy	连接状态
initMotorPosition	初始化电机位置
setMotorControlMode	设置 -- 自定义电机控制模式
setMotorMaxCurrent	设置 -- 自定义电机最大电流
setMotorMaxSpeed	设置 -- 自定义电机最大速度
enableMotor	使能 -- 手指电机使能
disableMotor	使能 -- 手指电机下使能
isMotorEnabled	使能 -- 手指电机使能状态
setJointPositionsAngle	控制 -- 手指关节角度控制
setJointPositionsRadian	控制 -- 手指关节弧度控制
setMotorTargetCurrent	控制 -- 电机目标电流
setMotorTargetSpeed	控制 -- 电机目标速度
calibrateSensor	标定 -- 灵巧手触觉传感器（校准操作）

帕西尼感知科技

getJointPositionsAngle	查询 -- 获取当前手指关节角度
getJointPositionsRadian	查询 -- 获取当前手指关节弧度
getMotorControlMode	查询 -- 控制模式
getFingerTactile	查询 -- 手指触觉传感器合力信息
getFingerTactileDetail	查询 -- 手指触觉传感器阵列力详细信息
getTemperature	查询 -- 传感器温度数据
getAllTactileData	查询 -- 全部数据（包括合力、阵列力和温度）
isFault	故障 -- 手指故障状态
getFaultCode	故障 -- 当前故障原因
clearFaultCode	故障 -- 清除故障码
getFrame	图像 -- 当前相机一帧图像
setCameraConfig	图像 -- 相机参数，包括分辨率和帧率
undistortImage	图像 -- 校正，去畸变的图像
getIntrinsicMatrixAndDistCoeffs	图像 -- 获取相机内参矩阵和畸变系数
getHandEyeMatrix	图像 -- 获取手眼相机外标定参数
getSDKVersion	版本 -- SDK 的版本信息
getFirmwareVersion	版本 -- 灵巧手固件版本信息

5.2 详细描述

5.2.1 类定义

初始化 DexH13Control 对象

帕西尼感知科技

灵巧手类名	构造函数
DexH13Control	DexH13Control();

5.2.2 灵巧手控制

表 7 灵巧手控制 Python 版详细接口说明

函数名 Function	参数 Parameters	返回值 Return	功能描述 Description
activeHandy(handy_port_num, camera_port_num = "none")	handy_port_num: 灵巧手的设备名称 camera_port_num : 相机是设备名称	成功：返回灵巧手的激活状态，1 表示激活左手，2 表示激活右手 失败：终止程序	扫描设备，激活灵巧手 Linux 系统下，灵巧手设备名可能是 /dev/ttyUSB0，相机设备名可能是 /dev/video0 或其他，请根据实际情况传参，只使用参数 1 时，相机默认不开
disconnectDevice()		返回 0	连接断开
isConnectHandy()		成功：返回 1 失败：返回值 0	连接状态
setMotorControlMode(ControlMode mode)	mode(ControlMode): 手指电机控制模式	成功：返回 1 失败：返回值 0	设置 -- 自定义手指电机控制模式，电机控制模式 ControlMode： NONE 无效值、TORQUE_CONTROL_MODE 力矩控制模式、SPEED_CONTROL_MODE 速度控制模式、

帕西尼感知科技

			POSITION_CONTROL_MODE 位置控制模式
setMotorMaxCurrent(max_current)	max_current: 手指电机的最大电流	成功：返回 1 失败：返回值 0	设置 -- 自定义手指电机最大电流，范围 [0,5], 单位 100mA, 例如 max_current=3，实际代表 300mA
setMotorMaxSpeed(max_speed)	max_speed: 电机的最大速度	成功：返回 1 失败：返回值 0	设置 -- 自定义电机最大速度，范围 [0,24000], 单位 r/min
enableMotor()		成功：返回 1 失败：返回值 0	使能 -- 手指电机使能，所有的灵巧手指电机
disableMotor()		成功：返回 1 失败：返回值 0	使能 -- 手指电机下使能，所有的灵巧手指电机
isMotorEnabled()		成功：返回 1 失败：返回值 0	使能 -- 手指电机使能状态，查询手指电机的使能状态
setJointPositionsAngle(joint_angles)	joint_angles(list)：角度对应 4 个手指 数组 1：食指角度 indexFinger 数组 2：中指角度 middleFinger 数组 3：环指角度 ringFinger 数组 4：大拇指角度 thumb	成功：返回 1 失败：返回值 0	控制 -- 手指关节角度控制，所有 FingerAngle 手指关节（食指、中指、环指、拇指）的关节空间范围： joint1 [-20°，+20°] 关节 1 侧摆角度（掌指双轴关节） joint2 [0°, 90°] 关节

帕西尼感知科技

			2 屈伸角度（掌指双轴关节） joint3 [0°, 90°] 关节 3 屈伸角度（近端关节） joint4 [0°, 90°] 关节 4 屈伸角度（末端关节）
setJointPositionsRadian(serial_rad)	serial_rad(list): 弧度对应 16 个关节限位	成功：返回 1 失败：返回值 0	控制 -- 手指关节弧度控制，所有手指关节的关节空间范围： joint1 [Pi/180° * -20°, Pi/180° * 20°]（掌指双轴关节-侧位摆动范围） joint2 [0, Pi/180° * 90°] (掌指双轴关节-弯曲伸展范围) joint3 [0, Pi/180° * 90°] (近端关节-弯曲伸展范围) joint4 [0, Pi/180° * 90°] (末端关节-弯曲伸展范围) 弧度公式：Pi/180° * 角度
setMotorTargetCurrent(target_current)	target_current(Dex13MotorCurrent): 电机的目标电流	成功：返回 1 失败：返回值 0	控制 -- 手指关节电流控制，设置手指所有关节电机的目标电流 目标电流范围：[-400, 400] 单位 mA, (峰值不能长时间使用) 一般使用时的范围为

帕西尼感知科技

			[-200, 200]
setMotorTargetSpeed(target_speed)	target_speed(Dex13MotorSpeed): 电机的目标速度	成功：返回 1 失败：返回值 0	控制 -- 手指关节速度控制，设置所有灵巧手指电机的目标速度 目标速度范围：[0, 24000]，单位 rpm （只在速度模式下需要设置）
calibrate_sensor()		成功：返回 True 失败：返回 False	标定 -- 灵巧手传感器（校准操作）

getMotorControlMode()		成功(ControlMode)：返回实际的控制模式
		失败：返回 0 无模式
			查询 -- 手指电机控制模式，enum ControlMode {
			NONE = 0,
			TORQUE_CONTROL_MODE = 1,
			SPEED_CONTROL_MODE = 2,
			POSITION_CONTROL_MODE = 3
			}

getJointPositionsAngle(joint_angles)		joint_angles(list): 角度对应 4 个手指 数组 1：食指角度 indexFinger 数组 2：中指角度 middleFinger 数组 3：环指角度 ringFinger	查询 --获取当前手指关节角度，所有 FingerAngle 手指关节（食指、中指、环指、拇指）的关节空间范围： joint1 [-20°，+20°] 关节 1 侧摆角度（掌指双轴关节） joint2 [0°, 90°] 关节 2 曲伸角度（掌指双

帕西尼感知科技

		数组 4：大拇指角度 thumb	轴关节） joint3 [0°, 90°] 关节 3 屈伸角度（近端关节） joint4 [0°, 90°] 关节 4 屈伸角度（末端关节）
getJointPositionsRadian(serial_rad)		serial_rad: 弧度对应 16 个关节限位	查询 -- 获取当前手指关节弧度， 所有手指关节的关节空间范围： joint1 [Pi/180° * -20°, Pi/180° * 20°]（掌指双轴关节-侧位摆动范围） joint2 [0, Pi/180° * 90°] (掌指双轴关节-弯曲伸展范围) joint3 [0, Pi/180° * 90°] (近端关节-弯曲伸展范围) joint4 [0, Pi/180° * 90°] (末端关节-弯曲伸展范围) 弧度公式：Pi/180° * 角度
isFault()		没有故障：返回 True 出现故障：返回 False	故障 -- 手指故障状态
getFaultCode();		返回当前出现的故障的错误码	故障 -- 当前故障原因，struct FingerMotorsError {std::string

帕西尼感知科技

		返回对象	fingers_name; // 手指名称 ErrorCode error_code; // 手指电机故障码，详见 define 文件定义；}
clearFaultCode()		成功：返回 True 失败：返回 False	故障 -- 清除故障码
getVersionSDK()		成功：返回 SDK 的版本信息 失败：返回空字符串	版本 -- SDK 的版本信息
getFirmwareVersion()		成功：返回灵巧手的固件版本信息 失败：返回空字符串	版本 -- 灵巧手的固件版本信息
initMotorPosition()		成功：返回 1 失败：返回 0	用于初始化电机位置，回到初始位置，建议在刚上电时调用，最大耗时约：5s

5.2.3 灵巧手相机

表 8 灵巧手相机 Python 版详细接口说明

函数名 Function	参数 Paramaters	返回值 Return	功能描述 Description
getFrame();		成功： 以 (numpy.array) 格式返回当前图	获取相机当前拍摄的一帧图像

帕西尼感知科技

		像帧。 失败: 返回空的(numpy.array)。	
setCameraConfig(width, height, fps);	1. width：图像宽度 2. height：图像高度 3. fps：帧率	成功: 返回 true。 失败: 返回 false，设置不生效。	设置相机参数，包括分辨率和帧率。只支持 640x480、1280x720、1920x1080 分辨率。 帧率只支持 30fps 以及 1920x1080 分辨率下的 60fps。
undistortImage(distortedFrame, cameraMatrix, distCoeffs);	1. distortedFrame：畸变图像(numpy.array) 2. cameraMatrix：相机内参矩阵(3x3 的 numpu.array) 3. distCoeffs：畸变系数(1x4 的 numpy.array)	成功: 返回去畸变的图像（numpy.array）。 失败: 返回空的。	图像去畸变。
getIntrinsicMatrixAndDistCoeffs (width, height);	1. width：图像宽度 2. height：图像高度	成功: 以元组的格式返回指定分辨率下的相机内参矩阵和畸变系数。其中内参矩阵是 3x3 的(numpy.array)，畸变系数是 1x4 的(numpy.array)	获取指定分辨率下相机内参矩阵和畸变系数。只支持 640x480、1280x720、1920x1080 分辨率。

柏恩感知科技

		。 失败: 终止程序	
getHandEyeMatrix();		成功: 返回相机手眼标定参数(numpy.array) 。 失败: 终止程序	获取相机手眼标定的转换矩阵（相机到灵巧手基座）。

5.2.4 灵巧手触觉传感器

表 9 灵巧手触觉 Python 版详细接口说明

函数名 Function	**参数 Paramaters**	**返回值 Return**	**功能描述 Description**
getFingerTactile()		成功: 返回手指触觉传感器的合力信息 失败: 返回空对象	查询 -- 获取手指触觉传感器信息，手有 11 个传感器，总共 11 个合力，每个传感器只有一个合力信息，合力包含 3 个数据，分别是 x、y、z ForcePoint: 手指触觉传感器信息，内部包含 3 个数据分别是 x: 传感器 x 轴数据 y: 传感器 y 轴

帕西尼感知科技

			数据 z: 传感器 z 轴数据
getFingerTactileDetail()		成功: 返回手指触觉传感器的阵列力信息 失败: 返回空对象	查询 -- 获取手指触觉传感器详细信息，手有 11 个传感器，总共 1140 分布力，分布力包含 3 个数据，分别是 x、y、z 其中食指中部，中指中部，环指中部的触觉传感器有 60 个分布力，其余的关节上触觉传感器有 120 个分布力 ForcePoint: 手指触觉传感器信息，内部包含 3 个数据分别是 x: 传感器 x 轴数据 y: 传感器 y 轴数据 z: 传感器 z 轴数据
getTemperature()		finger_tac(list): 手指触觉传感器	查询灵巧手的手指触觉传感

帕西尼感知科技

		的合力信息 finger_tac_detail (list): 手指触觉传感器的阵列力信息 temperature(list) : 手指触觉传感器的其他信息	器全部数据（其中，手指触觉传感器的其他信息的获取暂不支持）
getAllTactileData(finger_tac, finger_tac_detail, temperature)		成功：返回手指触觉传感器的温度信息 失败：返回空对象	查询 -- 传感器温度数据

六、示例程序

```
Python
#include <dexh13/dexh13_control.h>
#include <dexh13/dexh13_define.h>

#include <thread>

using namespace paxini::bot;
int main() {
  std::cout << "--- ----- --- Dexterous Hand Test Start --- -----
--- " << std::endl;
  std::string Test = "--- ----- ---";

  std::shared_ptr<paxini::bot::DexH13Control> dexh13 =
std::make_shared<paxini::bot::DexH13Control>();

  {
    // Connect the Hand
    dexh13->activeHandy("/dev/ttyUSB0", "none");

    // Check if the hand is connected
    bool sigConnected = dexh13->isConnectHandy();
```

```
    std::cout << Test << "isConnected: " << sigConnected <<
std::endl;
    if (!sigConnected) {
      throw std::runtime_error("isConnectHandy error");
    }
  }

  {
    //## Sensor calibration
    bool calibrate = dexh13->calibrateSensor();
    std::cout << Test << "calibrate: " << calibrate << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
  }

  int hz = 10;  // Set the query frequency

  while (true) {
    // # Obtain the resultant force data of the force sensor in
the X, Y, and Z directions
    std::vector<ForcePoint> dexh13FingerTac = dexh13-
>getFingerTactile();
    std::cout << Test << "dexh13 sensor data:" << std::endl;
    for (auto tac : dexh13FingerTac) {
      std::cout << "dexh13FingerTac x : " << tac.x << "y : " <<
tac.y << "z : " << tac.z << std::endl;
    }


std::this_thread::sleep_for(std::chrono::milliseconds(static_cast<
int>(1000.0 / hz)));
  }
}
```

详细示例请参见../example 路径，包含 c++及 python 版本示例。

七、常见问题

7.1 连接问题

1. 设备无法识别
    - 问题表现：在调用 `activeHandy` 接口进行连接时，系统提示找不到设备或连接失败。

- 可能原因：
  - 灵巧手未正确上电或电源连接不稳定。
  - 连接线缆损坏或接触不良。
  - 驱动程序未正确安装或版本不兼容。
- 解决方法：
  - 检查灵巧手的电源连接，确保电源线连接牢固且电源适配器正常工作。
  - 更换线缆，尝试使用其他线缆连接。
  - 重新安装或更新灵巧手的驱动程序，可从官方网站下载最新版本的驱动。

7.2 控制模式设置问题

1. 控制模式设置无效
   - 问题表现：调用 `setControlMode` 接口设置控制模式后，灵巧手的实际动作不符合设置的模式。
   - 可能原因：
     - 接口调用顺序错误，例如在未完成连接或初始化之前就设置控制模式。
     - 传入的控制模式参数不正确，可能是参数值超出范围或格式错误。
   - 解决方法：
     - 确保严格按照 SDK 的使用流程进行操作，先完成连接和初始化等前置步骤，并等待 6s 后，再设置控制模式。
     - 通过 `getControlMode` 接口仔细检查传入的参数，参考 SDK 文档中的参数说明，确保参数值正确无误。
2. 模式切换时出现异常动作
   - 问题表现：在切换不同的控制模式时，灵巧手出现抖动、卡顿或其他异常动作。
   - 可能原因：
     - 不同控制模式下的电机参数设置不一致，例如速度、电流限制等。
     - 切换过程中数据传输不及时或丢失。
   - 解决方法：
     - 在切换控制模式之前，先将电机参数恢复到默认值或根据新的模式要求进行合理设置。
     - 检查数据传输的稳定性，确保通信链路正常，可尝试降低数据传输速率或优

帕西尼感知科技

化通信协议参数。

7.3 电机控制问题

1. 电机无法使能
   - 问题表现：调用 `enableMotor` 接口后，电机没有响应，通过 `isMotorEnabled` 查询电机使能状态仍为未使能。
   - 可能原因：
     - 电机本身故障，例如绕组短路、断路等。
     - 电机驱动器故障或与灵巧手主板的连接问题。
   - 解决方法：
     - 使用万用表等工具检查电机的绕组电阻，判断电机是否正常。如果电机损坏，需要更换电机，请联系 paxini 售后支持。
     - 检查电机驱动器与主板之间的连接线是否插好，确保连接牢固。如果驱动器故障，请联系 paxini 售后支持；
2. 电机运动不精准
   - 问题表现：设置电机目标位置（`setMotorTargetPosition`）后，灵巧手的手指实际到达的位置与目标位置存在较大偏差。
   - 可能原因：
     - 传感器校准不准确，导致位置反馈数据有误。
     - 机械结构磨损或松动，影响了手指的运动精度。
   - 解决方法：
     - 调用 `calibrateSensor` 接口对传感器进行校准，确保位置反馈的准确性。
     - 检查灵巧手的机械结构，紧固松动的部件，如有磨损严重的零件，需要及时更换。

7.4 传感器数据问题

1. 传感器数据读取异常
   - 问题表现：通过 `getFingerTactile`、`getTemperature` 等接口读取传感器数据时，返回的数据为异常值（如超出正常范围的数值、恒定不变的值等）。
   - 可能原因：
     - 传感器本身故障或损坏。

帕西尼感知科技

- 传感器与主板之间的通信线路故障。

- 解决方法：
  - 首先检查传感器的连接线路，确保线路无断路、短路等问题。
  - 如果线路正常，可能是传感器故障，需要更换传感器并重新进行测试。
  - 当温度传感器温度获取为空，返回值为-0.10000000149011612，非异常结果。

2. 传感器数据更新延迟
   - 问题表现：传感器数据的更新频率明显低于预期，导致实时性较差，影响对灵巧手状态的监控和控制。
   - 可能原因：
     - 数据传输速率设置过低。
     - 系统资源占用过高，导致数据处理和传输延迟。
   - 解决方法：
     - 在 SDK 的配置中，适当提高数据传输速率，但要注意不要超过系统和硬件的支持能力。
     - 优化系统性能，关闭不必要的后台程序，释放系统资源，确保有足够的资源用于数据处理和传输。

以上常见问题，如需更换零部件请联系 paxini 售后支持团队。

7.5 图像处理问题

1. 图像获取失败
   - 问题表现：调用 `getFrame` 接口无法获取到相机的图像帧。
   - 可能原因：
     - 相机未正确连接或初始化。
     - 相机驱动程序问题。
     - 相机硬件故障。
   - 解决方法：
     - 检查相机的连接，确保电源线和数据线都连接良好。
     - 重新安装或更新相机驱动程序。
     - 尝试使用其他相机测试软件，判断相机是否正常工作，如果相机硬件故障，需要进行维修或更换。

帕西尼感知科技

2. 图像质量差
    - 问题表现：获取到的图像模糊、色彩失真或存在噪点等问题。
    - 可能原因：
        - 相机镜头脏污或有遮挡物；
        - 相机参数设置不合理，如帧率不适配移动速度等；
    - 解决方法：
        - 清洁相机镜头，移除遮挡物；
        - 通过 `setCameraConfig` 接口调整相机参数，根据实际环境和需求进行优化设置。

八、SDK 使用注意事项

1. 在使用 SDK 之前，请确保已经正确安装了 SDK 环境，并了解了灵巧手产品的基本操作和使用方法。
2. 在使用 SDK 时，请按照函数接口的说明进行调用，避免出现参数错误或调用顺序错误等问题。
3. 在控制灵巧手运动时，请根据实际情况设置合理的运动参数，避免出现运动失控或碰撞等问题。
4. 在获取传感器数据时，请根据实际情况设置合理的数据采集频率，避免出现数据丢失或延迟等问题。
5. 在使用 SDK 时，如果遇到问题或错误，请参考错误代码和错误信息进行排查和解决。

九、总结

本手册详细介绍了灵巧手产品 SDK 的使用方法和注意事项。通过使用 SDK，开发者可以轻松地控制灵巧手的运动、获取传感器数据，并实现各种应用场景。希望本手册能够对开发者有所帮助，使用户能够更好、更快的开发和应用灵巧手产品。

十、版权声明

1. 本文档会定期进行检查和修正，更新后的内容将出现在新版本中。本文档中的内

帕西尼感知科技

容或信息如有变更，恕不另行通知。

2. 使用产品前请阅读本文档。

3. 请保管好本文档，以便随时阅读与参考。

4. 本文档所记载的内容，不排除有错误或遗漏的可能性，如对本文档中的内容有疑问，请与我司联系 （https://www.paxini.com/）。

帕西尼感知科技